

파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

머릿말

Note. 참고 URL

https://github.com/seejokim1/NA_with_python/tree/main

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

Contents

1 파이썬의 이해	1
1.1 파이썬의 특징	1
1.1.1 플랫폼 독립성	1
1.1.2 인터프리터 언어	1
1.1.3 블록과 줄바꿈	1
1.1.4 객체지향 프로그래밍 지원	2
1.1.5 동적 타이핑	2
1.1.6 NumPy 라이브러리	3
1.2 변수선언	3
1.2.1 자료 형, 변수 선언과 변환	3
1.2.2 파이썬 예약어 (Keywords)	5
1.3 배열(Array)과 행렬(Matrix)	6
1.3.1 리스트(List)를 사용한 배열	6
1.3.2 리스트(List) 슬라이싱(Slicing)	8
1.3.3 NumPy와 배열과의 관계	9
1.3.4 행렬과 배열과의 관계	9
1.4 튜플(tuple)	10
1.4.1 튜플이란?	10
1.4.2 튜플(tuple) 형식	10
1.4.3 튜플(tuple) 특징	11
1.4.4 튜플(tuple) 예제	12
1.4.5 튜플(tuple) slicing : ":" 사용	13
1.4.6 튜플(tuple) Extended slicing : ":" 사용	13
1.5 사전(Dictionary)	14
1.5.1 딕셔너리(dictionary)	14
1.5.2 딕셔너리(dictionary) 예제	14
1.6 수학 연산자 및 NumPy 비교	15
1.6.1 파이썬의 기본 수학 연산자와 NumPy 연산 비교	15
1.6.2 Python 과 NumPy의 기본 연산	15
1.6.3 Python 과 NumPy의 배열 연산	16
1.7 반복 계산	18
1.7.1 반복계산이란?	18
1.8 조건 계산	23
1.8.1 if else 구문	23
1.8.2 다중 조건문 (if else else 구문)	24
1.8.3 if 구조 예문	24
1.9 입출력	27
1.9.1 키보드 입력 및 화면으로 출력	28

1.9.2	파일로 입력 및 파일로 출력	28
1.10	함수(Function)	31
1.10.1	내장함수와 사용자 정의 함수	31
1.10.2	파이썬과 NumPy의 내장함수 종류	31
1.10.3	사용자 정의 함수 선언과 특징	32
1.10.4	사용자 정의 함수 예제	33
1.11	NumPy를 이용한 벡터 행렬	35
1.11.1	내적(Dot Product)과 외적(Cross Product)	35
1.11.2	두 벡터간의 거리 구하기	36
1.11.3	두 행렬의 곱 (matrix multiplication)	37
1.12	객체지향 프로그래밍(OOP)	40
1.12.1	클래스 개요	40
1.12.2	클래스 선언	40
1.12.3	속성(변수)	40
1.12.4	클래스 메서드(함수)	40
1.12.5	객체지향 프로그래밍(OOP) 예제 1	41
1.12.6	객체지향 프로그래밍(OOP) 예제 2	42
1.13	matplotlib를 이용한 그래프 시각화	42
1.13.1	그래프 그리기 위한 주요 내장함수	43
1.13.2	2차원 그래프	43
1.13.3	2차원 산점도	44
1.13.4	3차원 그래프	44
1.14	연습문제	45

Chapter 1

파이썬의 이해

1.1 파이썬의 특징

1.1.1 플랫폼 독립성

파이썬 코드는 Windows, macOS, Linux 등 다양한 운영 체제에서 동일한 결과를 얻을 수 있다. 예를 들어, 간단한 파일 읽기 코드는 모든 운영 체제에서 동일하게 작동한다.

실습예제 https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_01_features.py

1.1.2 인터프리터 언어

파이썬 코드는 작성하면 바로 실행하여 결과를 확인할 수 있다. 파이썬 셸에서 간단한 계산을 바로 수행할 수 있다.

```
print(2 + 3)
# Output: 5
```

1.1.3 블록과 줄바꿈

파이썬에서 블록(block)은 들여쓰기(indentation)를 통해 정의된다. 파이썬은 중괄호 {} 대신 들여쓰기를 사용하여 코드의 구조를 나타낸다. 같은 수준으로 들여쓰기된 코드가 하나의 블록을 구성한다. 들여쓰기 규칙은 블록 내 코드는 4칸(space) 또는 탭(tab)으로 들여쓰면 된다. 같은 블록 내에서는 들여쓰기 크기를 일관되게 유지해야 한다. 서로 다른 들여쓰기는 오류를 발생시킨다. 블록이 끝나면 들여쓰기를 줄여야 하며, pass 키워드를 사용하여 빈 블록을 유지할 수 있다.

① 예제: 조건문과 반복문의 블록

코드	출력 결과
<pre> n = 5 if n > 0: # 조건문 블록 시작 print("n is positive") for i in range(n): # 반복 문 블록 시작 print(f"Iteration {i}") print("Loop finished") print("Outside all blocks") </pre>	<pre> n is positive Iteration 0 Iteration 1 Iteration 2 Iteration 3 Iteration 4 Loop finished Outside all blocks </pre>

1.1.4 객체지향 프로그래밍 지원

파이썬에서 클래스를 정의하고 객체를 생성하여 사용하는 방법은 객체지향 프로그래밍의 핵심 개념을 잘 보여준다.

코드	출력 결과
<pre> class Animal: def __init__(self, name): self.name = name def speak(self): return "makes a sound" class Dog(Animal): def speak(self): return "barks" pet = Dog("Buddy") print(pet.name, pet.speak()) </pre>	<pre> Buddy barks </pre>

1.1.5 동적 타이핑

파이썬에서는 변수에 대한 타입 선언 없이도 다양한 타입의 값을 할당하고 변경할 수 있다. 이는 코드 작성을 더욱 유연하고 간단하게 해준다.

코드	출력 결과
<pre>number = 10 # 처음에는 정 수를 할당 print(number) number = "ten" # 나중에는 문 자열을 할당 print(number)</pre>	<pre>10 ten</pre>

1.1.6 NumPy 라이브러리

NumPy는 파이썬에서 고성능 수치 계산을 가능하게 하는 라이브러리로, 대규모 다차원 배열과 행렬 연산에 최적화되어 있다. 이는 과학 및 공학 연산을 위한 강력한 도구로 자리 잡았다.

코드	출력 결과
<pre>import numpy as np a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) print(a + b) # 배열 간 덧셈</pre>	<pre>[5 7 9]</pre>

1.2 변수선언

1.2.1 자료 형, 변수 선언과 변환

자료형과 변수

파이썬은 동적 타이핑을 지원해 자료형 선언 없이도 변수를 사용할 수 있다. 문자열은 단일 문자도 문자열(str)로 처리하며, 숫자형과 문자열 간 변환이 간단하다. 예를 들어, `int("123")`은 문자열 숫자를 변환하고, `str(123)`은 정수를 문자열로 변환한다. 정수형(int)은 크기 제한 없이 동작하며, 실수형(float)은 부동 소수점 연산을 지원한다. 문자열은 다양한 연산을 제공하며 유니코드 기반으로 강력한 처리 기능을 가진다.

파이썬 메모리 관리

파이썬은 변수의 크기를 자동으로 조정하며, 정적 타입 언어처럼 메모리 크기를 명시하지 않아도 된다. 예를 들어, 정수의 경우 모두 사용자가 직접 메모리 관리를 고려하지 않아도 동작한다. 동적 메모리 관리로 인해 복잡한 저수준 문제를 처리하지 않아도 프로그램은 효율적으로 실행될 수 있다.

다양한 변수 선언에 대한 파이썬 코드와 실행 결과

이 표는 다양한 변수 선언과 그 결과를 명확히 설명하며, 파이썬의 변수 선언과 자료형 변환에 대한 이해를 돕는다. 각각의 사례는 파이썬의 동적 타이핑과 유연성을 강조하며, 프로그래밍 초보자에게 적합한 예이다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_02_variables.py

코드	출력 결과	설명
<pre>x = 10 print(x)</pre>	10	정수 값 10을 변수 x에 할당하였다. 파이썬은 값에 따라 자료형을 자동으로 결정한다. 여기서 x는 int 자료형으로 저장된다.
<pre>y = 3.14 print(y)</pre>	3.14	실수 값 3.14를 변수 y에 할당하였다. 파이썬은 이를 float 자료형으로 인식한다.
<pre>name = "Alice" print(name)</pre>	Alice	문자열 "Alice"를 변수 name에 할당하였다. 파이썬은 문자열을 str 자료형으로 처리한다.
<pre>is_valid = True print(is_valid)</pre>	True	Boolean 값 True를 변수 is_valid에 할당하였다. 이는 논리형으로 참을 나타낸다. 연산 시에는 값 1로 간주된다.
<pre>a = 5 b = a + 2 print(a, b)</pre>	5 7	변수 a에 정수 5를 저장하고, 변수 b에 a + 2의 결과를 저장하였다. 변수 간 연산이 정상적으로 수행되었음을 보여준다.
<pre>c = 10 c = "Hello" print(c)</pre>	Hello	변수 c에 처음에는 정수 10을 저장하고, 이후 문자열 "Hello"를 다시 저장하였다. 파이썬의 동적 타이핑 특징으로 같은 변수에 다른 자료형을 재할당할 수 있다.
<pre>try: print(a) except NameError as e: print(e)</pre>	name 'a' is not defined	선언되지 않은 변수 a에 접근하려고 시도하면 NameError가 발생한다. 이는 파이썬이 정의되지 않은 변수를 사용할 수 없도록 제한함을 보여준다.

코드	출력 결과	설명
<code>c = '76.3'</code>	없음	c는 문자열(str) 자료형으로 인식된다.
<code>d = float(c)</code>	없음	d는 실수형 자료형이 된다.
<code>print(c)</code>	76.3	문자열 c를 출력한다.
<code>print(d)</code>	76.3	실수형으로 변환된 d를 출력한다.
<code>print(c + d)</code>	오류 발생: TypeError	문자열과 숫자는 서로 연산할 수 없다.

코드	출력 결과	설명
<code>x = int(3.9)</code> <code>print(x)</code>	3	float 값을 int로 변환하여 소수점을 버리고 정수 부분만 저장한다.
<code>y = float(5)</code> <code>print(y)</code>	5.0	int 값을 float로 변환하여 소수점 아래에 0을 추가한 실수형으로 저장한다.
<code>z = str(42)</code> <code>print(z)</code>	42	결과는 문자열로, 숫자가 아닌 텍스트 데이터로 처리된다.
<code>a = "3.14"</code> <code>b = float(a)</code> <code>print(b)</code>	3.14	문자열로 저장된 숫자 값을 float로 변환한다. 이 경우, 문자열이 숫자 형식이어야 변환이 가능하다.
<code>c = "10"</code> <code>d = int(c)</code> <code>print(d)</code>	10	문자열 "10"을 정수형으로 변환하여 숫자 연산이 가능하게 만든다.
<code>e = 1 + True</code> <code>print(e)</code>	2	True는 숫자 1로 간주되어 정수 연산이 수행된다. 파이썬에서 True는 1, False는 0이다.

1.2.2 파이썬 예약어 (Keywords)

파이썬의 예약어(Keywords)는 언어의 문법과 구조를 정의하는 데 사용되는 미리 정의된 단어들이다. 이러한 예약어는 변수명이나 함수명으로 사용할 수 없으며, 각각 고유한 기능과 역할을 수행한다. 예를 들어, `if`, `else`, `for`는 조건문과 반복문을 정의하며, `class`, `def`는 클래스와 함수를 정의하는 데 사용된다.

또한, `import`는 외부 모듈을 가져오는 데 사용되고, `return`은 함수에서 값을 반환하는 데 활용된다. 이처럼 예약어는 파이썬 코드의 흐름과 동작을 제어하는 핵심 요소로, 총 33개의 예약어가 있으며 대소문자를 구분한다. 개발자는 예약어를 이해하고 적절히 사용하여 오류를 방지하고 가독성을 높일 수 있다.

Python Keywords (33 total):

and	as	assert	break	class
continue	def	del	elif	else
except	exec	finally	for	from
global	if	import	in	is
lambda	nonlocal	not	or	pass
print	raise	return	try	while
with	yield			

Remember: Keywords are case-sensitive!

1.3 배열(Array)과 행렬(Matrix)

1.3.1 리스트(List)를 사용한 배열

(1.3.1) 리스트(List)를 사용한 배열

파이썬에서 배열은 기본적으로 리스트로 구현된다. 리스트는 1차원 또는 다차원 데이터를 표현할 수 있으며, 자료형에 제한이 없어 다양한 데이터 타입을 혼합하여 저장할 수 있다. 배열은 파이썬에서 리스트(list)를 이용하여 간단히 생성할 수 있다. 예를 들어, 1차원 배열은 리스트 형태로 정의되며, 여러 데이터 타입을 저장할 수 있다.

리스트(List)는 파이썬에서 가장 기본적이고 유연한 데이터 구조 중 하나로, 데이터를 순서대로 저장할 수 있는 컬렉션이다. 리스트는 대괄호([])로 선언하며, 각 요소는 쉼표(,)로 구분한다.

리스트는 데이터 추가, 삭제, 수정 등 다양한 메서드를 제공하지만, 벡터 연산이나 행렬 연산 등 복잡한 수학적 계산에는 적합하지 않다. 대규모 데이터 연산에는 NumPy를 사용하는 것이 효율적이다.

① 리스트(list) 형식 및 특징

리스트는 다양한 자료형을 함께 저장할 수 있으며, 동적 크기 조정이 가능하다.

코드	출력 결과	설명
array = [1, 2, 3, 4] print(array)	[1, 2, 3, 4]	리스트를 사용하여 1차원 배열처럼 데이터를 저장하였다.
array_2d = [[1, 2], [3, 4]] print(array_2d)	[[1, 2], [3, 4]]	2차원 데이터를 생성하였다. 수학적 연산에는 적합하지 않다.
sum = array[0] + array[1] print(sum)	3	리스트 요소를 인덱싱하여 접근하고, 간단한 덧셈 연산을 수행하였다.

다양한 리스트 선언 예제

코드	출력 결과	설명
<code>colors = ['red', 'green'] print(colors)</code>	<code>['red', 'green']</code>	비어 있지 않은 리스트를 선언하였다.
<code>empty_list = [] print(empty_list)</code>	<code>[]</code>	비어 있는 리스트를 선언하였다.
<code>mixed_list = ['Hello', 1, 2] print(mixed_list)</code>	<code>['Hello', 1, 2]</code>	문자열과 숫자가 혼합된 리스트를 선언하였다.
<code>nest = [['Hello', 'World'], 1, 2] print(nest)</code>	<code>[['Hello', 'World'], 1, 2]</code>	리스트 안에 또 다른 리스트를 포함한 중첩 리스트를 선언하였다.

리스트 데이터 접근 예제

코드	출력 결과	설명
<code>a = [10, 20, 30, 40]print(a[0])</code>	10	첫 번째 요소 접근 (인덱스는 0부터 시작)
<code>print(a[-1])</code>	40	음수 인덱스를 이용하여 마지막 요소 접근
<code>sum = a[0] + a[1] print(sum)</code>	30	리스트 요소 간 연산 수행

중첩 리스트(Nested List)에서의 데이터 접근

코드	출력 결과	설명
<code>a = [1, 2, ['a', 'b', 'Life', 'is']] print(a[2][2][0])</code>	Life	인덱스를 중첩하여 원하는 데이터를 추출할 수 있다.

② 중첩 리스트의 행렬화

중첩 리스트는 행(row)과 열(column) 구조를 가지므로 행렬(matrix)로 간주할 수 있다.

$$A = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix}$$

코드	출력 결과	설명
<code>v1 = [1, 2, 3, 4, 5, 6, 7] v2 = [1, 2, 3, 4, 5, 6, 7] v3 = [1, 2, 3, 4, 5, 6, 7] A = [v1, v2, v3] print(A[0])</code>	<code>[1, 2, 3, 4, 5, 6, 7]</code>	리스트 v1, v2, v3를 생성하고, 이를 포함하는 2차원 리스트 A를 생성하였다. A[0]은 첫 번째 리스트 v1을 반환한다.
<code>print(A[1])</code>	<code>[1, 2, 3, 4, 5, 6, 7]</code>	두 번째 요소는 리스트 v2이다.
<code>print(A[2])</code>	<code>[1, 2, 3, 4, 5, 6, 7]</code>	세 번째 요소는 리스트 v3이다.

1.3.2 리스트(List) 슬라이싱(Slicing)

슬라이싱(Slicing)은 리스트에서 특정 범위의 요소를 쉽게 추출할 수 있는 강력한 기능이다. 슬라이싱을 사용하면 복잡한 반복문 없이 원하는 요소들을 빠르고 간결하게 가져올 수 있다. 이 기능은 데이터 분석, 배열 처리, 문자열 조작 등 다양한 분야에서 효율적으로 활용된다. 슬라이싱의 장점은 단순한 문법으로 간편한 구문으로 데이터를 추출 가능하며, 시작, 끝, 간격을 조절하여 다양한 데이터를 얻을 수 있는 유연성이 있다. 따라서 리스트의 특정 범위를 빠르게 추출할 수 있어 대규모 데이터 처리에도 적합하다.

① 슬라이싱의 필요성

- 효율적인 데이터 추출: 반복문 없이 특정 범위의 데이터를 쉽게 가져올 수 있다.
- 코드 간결성: 긴 코드를 줄이고 가독성을 높인다.
- 데이터 조작: 리스트의 일부를 다른 리스트로 저장하거나 수정할 수 있다.

② 슬라이싱의 기본 형식

기본 문법

```
list_name[start:end:step]
```

- **start**: 시작 인덱스(포함), 기본값은 0
- **end**: 끝 인덱스(포함하지 않음), 기본값은 리스트 길이
- **step**: 인덱스 간격(기본값은 1)

③ 코드 기초 예제

코드	출력 결과	설명
a = [0,1,2,3,4,5,6,7,8,9] print(a[2:5])	[2, 3, 4]	인덱스 2부터 4까지 요소 추출 (끝 인덱스 미포함)
print(a[:4])	[0, 1, 2, 3]	처음부터 3까지 요소 추출
print(a[5:])	[5, 6, 7, 8, 9]	인덱스 5부터 끝까지 추출
print(a[::2])	[0, 2, 4, 6, 8]	2칸 간격으로 요소 추출
print(a[-3:])	[7, 8, 9]	뒤에서부터 3개 요소 추출
print(a[::-1])	[9,8,7,6,5,4,3,2,1,0]	역순 추출

④ 데이터 분할

슬라이싱을 사용하여 데이터를 학습(train)과 테스트(test)로 나눌 수 있다.

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
train = data[:7] # 처음 7개
test = data[7:] # 나머지
print(train) # [1, 2, 3, 4, 5, 6, 7]
print(test) # [8, 9, 10]
```

⑤ 데이터 샘플링

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

sampled = a[::3] # 3칸 간격으로 추출

print(sampled) # [0, 3, 6, 9]
```

⑥ 리스트 복사

슬라이싱을 사용하면 리스트의 일부 또는 전체를 복사할 수 있다.

```
original = [1,2,3,4,5]
copied = original[:]

print(copied) # [1,2,3,4,5]
```

1.3.3 NumPy와 배열과의 관계

NumPy는 배열 연산에 특화된 라이브러리로, 리스트보다 메모리 효율적이고 연산 속도가 빠르다. NumPy 배열은 동일한 자료형만 저장 가능하며, 벡터화 연산을 통해 간단한 코드로 복잡한 연산을 수행할 수 있다.

NumPy 배열은 리스트보다 더 효율적이고 수학적으로 정교한 계산이 가능하다. 특히 벡터화 연산은 루프 없이 요소별 연산을 수행할 수 있어 코드가 간결해지고 성능이 향상된다. 리스트와 비교해 자료형 일관성과 연산 최적화 측면에서 큰 장점을 제공한다.

코드	출력 결과	설명
import numpy as np array_np = np.array([1, 2, 3, 4]) print(array_np)	[1, 2, 3, 4]	NumPy 배열을 생성. 리스트와 달리 동일한 자료형만 저장 가능하다. 수학적 연산에 적합.
array_2d = np.array([[1, 2], [3, 4]]) print(array_2d)	[[1, 2], [3, 4]]	2차원 데이터를 NumPy 배열로 표현하였다.
result = array_np + 10 print(result)	[11, 12, 13, 14]	벡터화 연산을 지원하여 각 요소에 10을 더하는 연산을 수행하였다.

1.3.4 행렬과 배열과의 관계

행렬은 데이터를 2차원으로 구조화한 형태로, 선형대수 연산에 적합하다. NumPy는 행렬 연산을 위한 강력한 기능을 제공하며, @ 연산자나 np.dot() 함수를 사용하여 행렬 곱과 같은 복잡한 연산을 간단히 처리할 수 있다. 행렬은 데이터를 2차원으로 구조화하여 다양한 연산에 적합하다. NumPy 배열은 행렬과 유사하지만, 기본 연산이 요소별(element-wise) 연산이라는 점에서 차이가 있다. 이를 통해 데이터 분석, 머신러닝 등 다양한 분야에서 활용된다. NumPy의 행렬 연산 기능은 복잡한 수학적 연산을 단순화하며 대규모 데이터를 효율적으로 처리할 수 있다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_03_arrays_matrices.py

코드	출력 결과	설명
<pre>A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]]) C = A @ B print(C)</pre>	<pre>[[19 22] [43 50]]</pre>	두 행렬 간의 행렬 곱을 수행하였다. @ 연산자는 NumPy에서 행렬 곱을 나타낸다.
<pre>transpose = A.T print(transpose)</pre>	<pre>[[1 3] [2 4]]</pre>	행렬의 전치를 수행하였다.
<pre>D = np.dot(A, B) print(D)</pre>	<pre>[[19 22] [43 50]]</pre>	np.dot() 함수는 행렬 곱 연산을 수행한다. @ 연산자와 동일한 결과를 출력한다.

1.4 튜플(tuple)

1.4.1 튜플이란?

리스트와 비슷한 순서형 자료형이지만, 리스트와 달리 생성된 항목을 변경할 수 없는 순서형이다. 이러한 성질 때문에 데이터의 수정이 필요가 없거나 데이터의 안정성을 보장해야 할 때 튜플을 사용한다. 데이터를 중앙으로 변경해야 하는 경우에는 리스트를 사용하는 것이 좋다. 또한, 문법적으로 리스트는 대괄호([])를 사용하여 생성하고, 튜플은 소괄호(())를 사용하여 생성한다.

1.4.2 튜플(tuple) 형식

튜플의 형식은, 리스트에서는 대괄호 “[]” 를 이용하여 만들던 튜플을 소괄호 “()” 를 이용하여 만든다. () 괄호 사용. 항목 추출하기는 리스트처럼 숫자 인덱스를 사용하거나 콤마(,) 를 사용한다. 파이썬 튜플은 리스트와 같이 순서가 있지만, 수정 불가능한 자료구조이다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_04_tuples.py

코드	출력 결과
<pre>T = ((10, 20), (30, 40), (50, 60)) print(T)</pre>	<pre>T = ((10, 20), (30, 40), (50, 60))</pre>
<pre>print(T[0][0])</pre>	<pre>T[0][0] = 10</pre>
<pre>T4 = (1, 2, ('a', 'b', ('Life', 4))) print(T4[2][2][0])</pre>	<pre>Life</pre>

1.4.3 튜플(tuple) 특징

튜플의 요소 값은 변경하거나 삭제할 수 없다.

코드	출력 결과
<pre>T = ((10, 20), (30, 40)) T[0][0] = 55</pre>	<pre>TypeError Traceback (most recent call last) Cell In[3], line 2 1 T = ((10, 20), (30, 40)) ----> 2 T[0][0] = 55 TypeError: 'tuple' object does not support item assignment</pre>

값이 하나만 있는 튜플은 콤마 하나 꼭 붙여야 한다.

코드	출력 결과
<pre>T = (55,) print(T[0])</pre>	<pre>55</pre>

코드	출력 결과
<pre>T = (55) print(T[0])</pre>	<pre>TypeError Traceback (most recent call last) Cell In[5], line 2 1 T = (55) ----> 2 print(T[0]) TypeError: 'int' object is not subscriptable</pre>

1.4.4 튜플(tuple) 예제

코드	출력 결과
<pre>my_list = [1, 2, 3, 4, 5] 리스트 생성 print(my_list, my_list)</pre>	<pre>my_list [1, 2, 3, 4, 5]</pre>
<pre>my_tuple = (1, 2, 3, 4, 5) 튜플 생성 print(my_tuple, my_list)</pre>	<pre>my_tuple (1, 2, 3, 4, 5)</pre>
<pre>my_list[0] = 6 리스트 값 변경 print("my_list=", my_list)</pre>	<pre>my_list= [6, 2, 3, 4, 5]</pre>
<pre>my_list.append(6) 리스트 값 추가 print(my_list.append(6), my_list)</pre>	<pre>None [6, 2, 3, 4, 5, 6]</pre>
<pre>my_tuple[0] = 6 튜플 값 변경 - 오류 발생 print("my_tuple[0]", my_list) Cell In[6], line 18 --> my_tuple[0] = 6 TypeError: 'tuple' object does not support item assignment</pre>	<pre>TypeError Traceback (most recent call last)</pre>
<pre>my_tuple.append(6) 튜플 값 추가 - 오류 발생 print("my_tuple.append(6)", my_list)</pre>	<pre>TypeError: 'tuple' object has no attribute 'append'</pre>

1.4.5 튜플(tuple) slicing : ":" 사용

코드	출력 결과
<pre> T = (1, 2, 3, 4, 5, 6, 7) T1 = T[0:3] ; T2 = T[0:-3] T3 = T[:] ; T4 = T[5:] ; T5 = T[:5] print('T :', T) print('T[0:3] :', T1) print('T[0:-3] :', T2) print('T[:]' :', T3) print('T[5:] :', T4) print('T[:5] :', T5) T[0:3] : (1, 2, 3) T[0:-3] : (1, 2, 3, 4) T[:] : (1, 2, 3, 4, 5, 6, 7) T[5:] : (6, 7) T[:5] : (1, 2, 3, 4, 5) </pre>	<pre> T : (1, 2, 3, 4, 5, 6, 7) </pre>

1.4.6 튜플(tuple) Extended slicing : ":" 사용

A[:], A[1:2:3], A[::-1] 등으로 배열의 index에 접근하는 방법이다. A(a:b:c)의 의미는 index a부터 index b까지 c의 간격으로 배열을 만드는 것이다. 만약 A가 없으면, 즉 A(:b:c) 이라면 처음부터 b까지 c의 간격으로 배열을 만드는 것이다. A가 없으면, 즉 A(a::c) 이라면 index a부터 끝까지 c의 간격으로 배열을 만드는 것이다. 예를 들어 a가 양수라면 마지막 index까지, c가 음수라면 첫 index까지 거꾸로 배열을 만드는 것이다. 마지막으로 a가 없으면, 즉 A(::c) 이라면 index 0부터 index b까지 c의 간격으로 배열을 만드는 것이다. Extended slicing ":" 사용하여 파이썬 프로그램을 해 보자.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_04_06_tuple_extended_slicing.py

코드	출력 결과
<pre> A = [0, 1, 2, 3, 4, 5, 6] A_copy = A[:] # A[:] - 배열 전체를 복사 # A[1:2:3] 인덱스 1부터 2까지 3의 간격으로 추출 A_slice_1 = A[1:2:3] A_reverse = A[::-1] # A[::-1] - 배열 을 역순으로 추출 print(f"A_copy: {A_copy}") print(f"A_slice_1: {A_slice_1}") print(f"A_reverse: {A_reverse}") A_slice_1: [1] A_reverse: [6, 5, 4, 3, 2, 1, 0] </pre>	<pre> A_copy: [0, 1, 2, 3, 4, 5, 6] </pre>

1.5 사전(Dictionary)

1.5.1 딕셔너리(dictionary)

- 16 -

사전 (Dictionary)

딕셔너리는 키(key)와 값(value)의 쌍으로 데이터를 저장하는 자료구조로, 데이터를 빠르게 검색하거나 조작할 때 유용하다. 딕셔너리는 중괄호 {}를 사용하며, 키는 고유해야 하고 변경 불가능한 자료형 (예: 문자열, 숫자)을 사용한다. 값은 어떠한 자료형도 가능하다. 딕셔너리는 데이터를 추가, 삭제, 업데이트와 같은 작업을 효율적으로 처리한다. 데이터를 체계적으로 저장하고 검색하는 데 적합하다.

딕셔너리는 다음과 같은 특징을 가진다.

1. **키(key):** 변경 불가능한(immutable) 데이터 타입만 사용할 수 있다. 예: 문자열, 숫자, 튜플.
2. **값(value):** 모든 데이터 타입을 허용한다.
3. **속도:** 데이터 검색과 수정 속도가 빠르다.
4. **가변성:** 딕셔너리는 동적으로 수정, 추가, 삭제가 가능하다.

1.5.2 딕셔너리(dictionary) 예제

실습 코드: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_05_dictionary.py

```

# Dictionary creation
student = {
    "name": "Alice",
    "age": 25,
    "major": "Engineering"
}

# Value access
print(student["name"]) # Output: Alice

# Value modification
student["age"] = 26

# Adding new key-value pair
student["grade"] = "A"

# Deleting a key
del student["major"]

print(student) # {'name': 'Alice', 'age': 26, 'grade': 'A'}

```

1.6 수학 연산자 및 NumPy 비교

1.6.1 파이썬의 기본 수학 연산자와 NumPy 연산 비교

스칼라 산술 연산과 요소 순서연산이다. 곱(*), 제곱(**), 나눗셈(/) 등의 기본 연산은 파이썬에서 직접 사용할 수 있다. 정수형 나눗셈은 몫과 나머지를 구분한다. 예를 들어 몫을 구하려면 // 연산자를 사용하고, 나머지를 구하려면 % 연산자를 사용한다. 나머지를 출력하려면 정수에 %를 입력하고 나누려는 값을 입력하면 된다.

Operation	Python	NumPy
Addition	a + b	np.add(a, b)
Multiplication	a * b	np.multiply(a, b)
Division	a / b	np.divide(a, b)
Power	a ** b	np.power(a, b)
Square root	-	np.sqrt(a)
Matrix mult.	-	np.dot(a, b) or a @ b

1.6.2 Python 과 NumPy의 기본 연산

실습 코드: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_06_operators_numpy.py

1.6.3 Python 과 NumPy의 배열 연산

Python의 기본 연산자는 리스트를 사용할 때 요소별 연산을 자동으로 수행하지 않는다. 대신, 반복문이나 리스트 컴프리헨션을 사용해야 한다.

코드	출력 결과
<pre># Python 기본 연산 (리스트) a = [1, 2, 3]; b = [4, 5, 6] c = [a[i] + b[i] for i in range(len(a))] print(c) # 출력: [5, 7, 9]</pre>	[5, 7, 9]

NumPy는 벡터 연산을 지원하여 반복문 없이도 배열의 요소별 연산을 간단히 수행할 수 있다.

코드	출력 결과
<pre># NumPy 배열 연산 import numpy as np a = np.array([1, 2, 3]); b = np.array([4, 5, 6]) c = a + b; print(c) # 출력: [5, 7, 9]</pre>	[5, 7, 9]

Python lists vs NumPy arrays for large-scale operations:

```
import numpy as np
import time

# Python list operation
a = list(range(1_000_000))
b = list(range(1_000_000))
start = time.time()
c = [a[i] + b[i] for i in range(len(a))]
print("List operation:", time.time() - start)

# NumPy array operation
a_np = np.arange(1_000_000)
b_np = np.arange(1_000_000)
start = time.time()
c_np = a_np + b_np
print("NumPy operation:", time.time() - start)
```

Result: NumPy is significantly faster (often 10-100x) for numerical operations!

Python 리스트의 연산에서는 더하기(+)나 반복하기(*) 같은 연산자가 적용된다. 여기서 덧셈, 곱셈은 원래 수학적 연산이 아니라 다른 의미의 연산임을 주의하자. 덧셈 연산은 벡터의 각 요소를 더하는 것이 아니라 리스트에 새로운 데이터를 덧붙이는 연산자는 반복하는 연산자는 반복하기이다. 리스트 크기가 달라진다. 따라서 python에서 수치해석용으로 활용할 때는 numpy를 사용하는 것이 바람직하다.

코드	출력 결과
<pre>v1 = [1, 2, 3] v1 * 3</pre>	<pre>[1, 2, 3, 1, 2, 3, 1, 2, 3]</pre>

NumPy에서도 단일 값 연산이 가능하지만, 주로 배열과 함께 사용한다. numpy를 사용한 스칼라(scalar), 벡터(vector) 연산의 예는 아래와 같다.

numpy를 사용한 벡터(vector) + 연산:

코드	출력 결과
<pre>import numpy as np v1 = np.array([1,2,3]) v2 = np.array([1,2,3]) print(v1 + v2)</pre>	<pre>[2 4 6]</pre>

numpy를 사용한 벡터(vector) 스칼라 곱하기 연산:

코드	출력 결과
<pre>import numpy as np v1 = np.array([1,2,3]) v2 = np.array([1,2,3]) print(3*v1 + 2*v2)</pre>	<pre>[5 10 15]</pre>

Python 기본 연산자는 행렬 곱 연산을 지원하지 않는다. NumPy는 @ 연산자 또는 np.dot()을 사용하여 행렬 곱을 수행할 수 있다. numpy를 사용한 두 벡터(vector) 내적 계산은 sum(v1*v2) 또는 np.dot(v1,v2)를 사용하면 된다. matrix [A], matrix [B] multiplication 경우 np.dot(A,B)를 사용한다.

코드	출력 결과
import numpy as np	
v1 = np.array([1,2,3])	
v2 = np.array([1,2,3])	
print(sum(v1*v2))	14
print(np.dot(v1,v2))	14

1.7 반복 계산

1.7.1 반복계산이란?

반복 계산은 동일한 연산을 여러 번 수행하여 결과를 얻는 과정으로, 수학적 문제 해결, 데이터 처리, 과학적 계산 등에서 필수적이다. Python은 반복문(for, while)과 벡터화 연산을 통해 효율적인 반복 계산을 구현할 수 있으며, 이는 대규모 데이터 처리와 복잡한 알고리즘 설계에 중요한 역할을 한다. 반복문을 이해하고 이를 적절히 활용하는 능력은 프로그래밍의 핵심 역량이다. 특히, 큰 행렬의 계산이나 반복 구조를 처리하는 과정을 자동화하여 효율성을 높인다.

① 특징:

C/C++/Matlab/Java 언어처럼 파이썬에서도 for 문으로 반복 계산이 이루어진다. if, 서술문이나 기타 들여쓰기 된 블록처럼 for 문도 얼마든지 그 안에 포함된 코드를 작성할 수 있다.

② for 구문:

for statement는 반복문 중 하나로, 주어진 iterable(반복 가능한) 객체의 모든 요소에 대해 반복하여 작업을 수행하는 데 사용된다. for 문은 주로 리스트, 튜플, 딕셔너리, 문자열과 같은 iterable(반복 가능한) 객체를 다룰 때 많이 사용된다. for 문은 다음과 같은 구조를 가지게 된다. variable은 반복문에서 현재 처리 중인 요소(element)를 나타내는 변수, iterable은 반복문이 실행될 iterable 객체. 만약 특정 횟수만큼 실행시키고 싶다면, 목록 부분에 range() 함수를 사용하면 된다.

```
for variable in iterable:
```

③ 범위(range)를 사용:

for 문을 사용할 때, 범위(range)를 사용하여 어떻게 변수가 추출되어 반복 계산하는지 아래 예를 들어 설명한다. 반복문을 사용하였다. 시작에 for 입력하여 그 뒤의 문자를 반복해서, 반복문을 사용될 때 range라는 범위를 입력하면 범위의 숫자를 반복해서 계산하게 된다. 만약 range(1,4), 라면 1에서 4미만의 수를 반복하여 사용한다. 만약 1부터 4까지의 숫자를 사용하고 싶다면 range(1,5) 또는 range(0,4)를 사용하여 된다.

주석으로 설명한 소스 코드	출력 결과
<pre># 첫 번째 for 루프: range() 함수를 사용하여 1부터 3까지 반복 # range(1, 4)는 1, 2, 3을 생성 (4는 포함되지 않음) for val in range(1, 4): print(val) # 현재 반복 중(val)을 출력</pre>	<pre>1 2 3</pre>
<pre># 두 번째 for 루프: 리스트 [1, 2, 3, 4]를 순회 # 리스트의 각 요소를 변수 i에 할당 for i in [1, 2, 3, 4]: print(i) # 현재 리스트 요소(i)를 출력</pre>	<pre>1 2 3 4</pre>

④ 반복문을 이용한 합 계산

for 반복문을 사용하여 1부터 10까지의 숫자를 순차적으로 더했다. 이 방식은 간단한 반복 계산에 적합하며, 원하는 범위 내에서 반복 연산을 수행한다.

주석으로 설명한 소스 코드	출력 결과
<pre># 1부터 10까지의 합 계산 total = 0 for i in range(1, 11): total += i print("1부터 10까지의 합:", total)</pre>	<pre>1 부터 10 까 지 의 합: 55</pre>

⑤ 리스트를 이용한 반복 계산

리스트에 저장된 각 요소를 반복문을 통해 제공한 뒤 새로운 리스트에 저장하였다. 반복 계산은 리스트와 같은 데이터 구조에서도 간단히 구현할 수 있다.

주석으로 설명한 소스 코드	출력 결과
<pre># 리스트 요소의 제곱 계산 numbers = [1, 2, 3, 4, 5] squared = [] for num in numbers: squared.append(num ** 2) print("리스트의 제곱:", squared)</pre>	<pre>리 스톱 의 제 곱: [1, 4, 9, 16, 25]</pre>

⑥ 예제: 0부터 n까지의 홀수 합

1부터 n까지의 숫자를 다 더하고 싶다면 문자 하나를 0으로 정의해 놓고 for 반복문에 범위를 range(1,n+1)로 입력한다. 그런 다음 0으로 정의해 놓은 문자에 반복되는 문자를 더해주면 1부터 n까지의 합을 구할 수 있다. 여기에서는 0부터 n까지 홀수의 합을 계산하는 프로그램을 만들어보자. n=100인 경우 그 결과는 어떻게 되는지 확인해본다.

주석으로 설명한 소스 코드	출력 결과
<pre> print("0부터 n까지의 홀수 합을 계산.") print("n을 입력하세요.") # 입력받은 값을 정수로 변환하여 변수 n에 저장 n = int(input()) # 합계를 저장할 변수 초기화 val = 0 # 0부터 n까지 반복 # i는 0부터 n까지 순서대로 값을 가짐 # i가 홀수 일 때만 더하기 (i % 2 == 1 이면 홀수) for i in range(0, n + 1): if i % 2 == 1: val = val + i # 홀수 일 때 val에 i를 더함 print("n까지의 홀수 합 :", val) </pre>	<pre> 0 부터 n 까 지 의 홀 수 합 을 계산. n 을 입 력 하 세 요. 100 n 까 지 의 홀 수 합 : 2500 </pre>

⑦ 예제: 0부터 n까지의 짝수 합

0에서 n까지 짝수 합을 계산하는 함수를 만들고 n=100인 경우 그 결과는 어떻게 되는가?

주석으로 설명한 소스 코드	출력 결과
<pre> # 사용자에게 알림 메시지 출력 print("0부터 n까지의 짝수 합을 계산.") # 사용자로부터 n 값을 입력받음 print("n을 입력하세요.") # 입력받은 값을 정수로 변환하여 변수 n에 저장 n = int(input()) # 합계를 저장할 변수 초기화 val = 0 # 0부터 n까지 반복 # i는 0부터 n까지 순서대로 값을 가짐 # i가 짝수(i % 2로 나눈 나머지가 0이면 짝수) for i in range(0, n + 1): if i % 2 == 0: val = val + i # 짝수일 때 val #에 i를 더함 # 계산된 짝수 합을 출력 print("n까지의 짝수 합 :", val) </pre>	<pre> 0 부터 n 까 지 의 짝 수 합 을 계 산 . n 을 입 력 하 세 요 . 100 n 까 지 의 짝 수 합 : 2550 </pre>

⑧ 예제: 함수값 이용

반복문을 사용하여 함수값을 연속적으로 구해보자. 임의로 정의한 $f(x) = x^2$ 에 값을 0부터 4까지의 수를 대입하여 구하기 위해서는 값에 들어갈 숫자의 범위를 정해준 뒤 반복한다. 프로그램을 프린트하면 4개가 숫자를 차례대로 넣은 함수값이 나오게 된다. 변수 x 의 값을 정해준 뒤 다음 range 용어를 활용하여 값 도출하는 것이다. range 함수를 통해 반복문에 수를 차례 반복하여 $f(x) = x^2$ 값을 계산한다.

주석으로 설명한 소스 코드	출력 결과
<pre># 함수 정의: 입력값 x를 제공하여 반환하 는 함수 f(x) def f(x): return x**2 # x를 제공한 값을 반 환 # 반복문: range(0, 4)를 이용하여 0부 터 3까지 반복 for x in range(0, 4): # x 는 0, 1, 2, 3의 값을 순서대로 가짐 print(f(x)) # 함수 f(x)의 결 과 (제공값)를 출력</pre>	<pre>0 1 4 9</pre>

(1.7.3) while 문을 이용한 반복 계산

while 문은 조건이 참(True)인 동안 계속해서 반복 실행되는 루프를 제공한다. 조건이 거짓(False)이 되면 루프가 종료된다. 주로 반복 횟수가 명확하지 않거나, 특정 조건을 만족할 때까지 실행이 필요한 경우에 사용된다. 조건식은 반복 실행자를 결정하는 논리식이다. True일 때 반복이 진행된다. 반복 실행할 코드: 조건식이 참일 때 실행되는 코드를 블록이다.

while 조건식:

반복 실행할 코드

이 코드의 작동 방식은 사용자가 입력한 값 n 까지 1부터 반복하면서 합계를 계산한다. while 문의 조건이 $i \leq n$ 일 때 루프가 반복되므로, i 가 n 을 초과하면 루프가 종료된다. 이 방식은 조건 기반 반복의 전형적인 사용 예이다.

주석으로 설명한 소스 코드	출력 결과
<pre># 1부터 n까지의 합을 계산 # 프로그램 시작 메시지 출력 print("1부터 n까지의 합을 계산") # 사용자로부터 n 값을 입력받음 # 입력값을 정수로 변환하여 n에 저장 n = int(input("n 값을 입력: ")) # 반복에 필요한 초기 변수 설정 i = 1 # 반복 변수, 1부터 시작 total = 0 # 합계를 저장할 변수 # while 문을 이용한 반복 # i가 n 이하일 때 루프를 실행 # total 변수에 i 값을 더함 (누적 계산) # 반복 변수를 1 증가시킴 (루프 탈출 조건 형성) while i <= n: total += i i += 1 # 최종 결과 출력 print(f"1부터 {n}까지의 합은 {total}.")</pre>	<pre>1부터 n까지의 합을 계산. n 값을 입력: 100 1부터 100까지의 합은 5050.</pre>

1.8 조건 계산

1.8.1 if else 구문

if else 구문은 조건에 따라 프로그램이 서로 다른 코드를 실행하도록 제어하는 조건문이다. if 문은 조건이 참(True)일 경우 실행되는 코드 블록을 지정하고, else 문은 if 조건이 거짓(False)일 경우 실행되는 대체 코드를 지정한다. 함수 안의 코드 블록은 들여쓰기로 표시한다. 프로그래밍 언어에 따라 들여쓰기를 블록의 실행여부로 구분하거나 중괄호 {}로 블록을 구분한다. C/C++/Java 등의 언어는 {}를 사용하여 블록을 정의하고, Matlab은 if, elseif, else, end 구조를 가진다. Python은 들여쓰기를 통해 코드 블록을 구분하며, 들여쓰기 수준이 같은 코드만 동일 블록으로 인식된다.

if 조건식:

 # 조건이 참일 때 실행할 코드

else:

 # 조건이 거짓일 때 실행할 코드

1.8.2 다중 조건문 (if else else 구문)

if elif else 구문은 다중 조건을 처리하기 위해 사용된다. 여러 조건 중 하나만 만족하면 해당 블록의 코드가 실행되고, 이후 조건 검사는 중단된다.

```
if 조건식1:
    # 조건식1이 참일 때 실행할 코드

elif 조건식2:
    # 조건식1이 거짓이고, 조건식2가 참일 때 실행할 코드

elif 조건식3:
    # 조건식1과 조건식2가 거짓이고, 조건식3이 참일 때 실행할 코드

else:
    # 위의 모든 조건이 거짓일 때 실행할 코드
```

1.8.3 if 구조 예문

실습 코드: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_08_conditionals.py

① 예제 코드: 숫자가 짝수인지 홀수인지 판별하기

주석으로 설명한 소스 코드	출력 결과
<pre># 사용자로부터 숫자를 입력받음 # 입력값을 정수로 변환 num = int(input("숫자를 입력하세요: ")) # if ~ else 구문을 이용해 짝수와 홀수 판별 # 숫자를 2로 나눈 나머지가 0이면 짝수 if num % 2 == 0: print(f"{num}은 짝수입니다.") else: print(f"{num}은 홀수입니다.")</pre>	<pre>숫자를 입력하세요: 5 5은 홀수입니다.</pre>

② 리스트와 조건문을 이용한 다중 조건 계산

Python에서는 리스트와 조건문을 결합하여 여러 데이터를 효율적으로 처리할 수 있다. 리스트 `scores`에 있는 각 점수를 조건문으로 평가하여 등급(`grades`) 리스트에 저장하였다. 반복문과 조건문을 조합하여 다중 조건 계산을 수행하는 예제이다.

주석으로 설명한 소스 코드	출력 결과
<pre># 학생 점수와 등급 계산 scores = [95, 82, 67, 58, 89] grades = [] for score in scores: if score >= 90: grades.append('A') elif score >= 80: grades.append('B') elif score >= 70: grades.append('C') else: grades.append('D') print("점수:", scores) print("등급:", grades)</pre>	<pre>점 수: [95, 82, 67, 58, 89] 등 급: ['A', 'B', 'D', 'D', 'B']</pre>

③ 조건 표현식을 이용한 간단한 조건 계산

Python은 조건 표현식(삼항 연산자)을 사용하여 간결한 조건 계산을 구현할 수 있다. 조건 표현식은 상황에 따라 값을 반환할 수 있으며, 조건을 한 줄로 표현할 수 있다. 단일 라인으로 조건 계산을 구현할 때 매우 유용하다.

주석으로 설명한 소스 코드	출력 결과
<pre># 숫자가 양수, 음수 또는 0인지 판 별 number = -5 result = " 수" if number > 0 else " 수" if number < 0 else "0" print(f" 숫 자: {number}, 결 과: {result}")</pre>	<pre>숫 자: -5, 결 과: 음 수</pre>

(1.8.4) 비교연산자

비교 연산자는 두 값을 비교하여 참(True) 또는 거짓(False)을 반환한다. 이 연산자는 조건문이나 반복문에서 조건을 설정하는 데 주로 사용된다. 주요 비교 연산자는 다음과 같다.

- == : 값이 동일한지 확인한다.
- != : 값이 서로 다른지 확인한다.
- > : 왼쪽 값이 오른쪽 값보다 큰지 확인한다.
- < : 왼쪽 값이 오른쪽 값보다 작은지 확인한다.
- >= : 왼쪽 값이 오른쪽 값보다 크거나 같은지 확인한다.
- <= : 왼쪽 값이 오른쪽 값보다 작거나 같은지 확인한다.

이러한 비교 연산자는 조건문(if)과 반복문(while)에서 조건을 설정할 때 자주 사용된다.

Operator	Meaning	Example	Result
==	Equal	5 == 5	True
!=	Not equal	5 != 3	True
>	Greater than	5 > 3	True
>=	Greater or equal	5 >= 5	True
<	Less than	3 < 5	True
<=	Less or equal	3 <= 5	True

Usage:

- Conditional statements (if, elif, else)
- While loops
- Boolean expressions

① 예제 코드: 비교 연산자 사용

주석으로 설명한 소스 코드	출력 결과
두 숫자를 입력받아 비교 <pre>a = int(input("첫 번째 숫자를 입력: ")) b = int(input("두 번째 숫자를 입력: "))</pre> a와 b가 같은지 비교 <pre>print(f"{a} == {b}: {a == b}")</pre> a와 b가 다른지 비교 <pre>print(f"{a} != {b}: {a != b}")</pre> a가 b보다 큰지 비교 <pre>print(f"{a} > {b}: {a > b}")</pre> a가 b보다 크거나 같은지 비교 <pre>print(f"{a} >= {b}: {a >= b}")</pre> a가 b보다 작은지 비교 <pre>print(f"{a} < {b}: {a < b}")</pre> a가 b보다 작거나 같은지 비교 <pre>print(f"{a} <= {b}: {a <= b}")</pre> 두 번째 숫자를 입력: 5 <pre>1 == 5: False 1 != 5: True 1 > 5: False 1 >= 5: False 1 < 5: True 1 <= 5: True</pre>	첫 번째 숫자를 입력: 1

1.9 입출력

(1.9.1) 키보드로 입력 및 화면으로 출력

키보드 입력과 화면 출력은 Python의 `input()` 함수는 사용자가 입력한 값을 문자열로 반환한다. `input()`으로 사용자의 이름과 나이를 입력받고, `print()`로 나이, 이름을 출력할 수 있다. 사용자와 상호작용을 수행하기 위해 `input()`을 사용하여 명시적으로 값을 반환받고 출력할 수 있다.

1.9.1 키보드 입력 및 화면으로 출력

https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_09_io.py

Overview

This section explains methods for handling data input and output in Python.

Keyboard Input and Screen Output:

```
# Input from user
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# Output to screen
print(f"Hello, {name}.")
print(f"You are {age} years old.")

# Example:
# Enter your name: Seejo Kim
# Enter your age: 22
# Output: Hello, Seejo Kim.
#         You are 22 years old.
```

1.9.2 파일로 입력 및 파일로 출력

Python의 `open()` 함수를 사용하여 파일 입출력을 쉽게 구현할 수 있다. 파일 모드는 다음과 같다.

- w: 쓰기 모드 (파일이 존재하지 않으면 새로 생성)
- r: 읽기 모드
- a: 추가 모드 (기존 내용에 추가)

`with open()` 구문은 파일을 자동으로 열고 닫아주는 안전한 방법이다. `write()`를 사용하여 파일에 데이터를 저장하고, `read()`를 사용하여 파일 내용을 읽는다.

① 파일 쓰기

주석으로 설명한 소스 코드	출력 결과
<pre># 파일에 쓰기 with open("output.txt", "w") as file: file.write("안녕하세요.\n") file.write("Python 파일 입출력 예제입니다.\n") Python 파일 입출력 예제입니다.</pre>	안녕하세요.

② 파일 읽기

주석으로 설명한 소스 코드	출력 결과
<pre># 파일 읽기 with open("output.txt", "r") as file: content = file.read() print("파일 내용:\n", content) 안녕하세요. Python 파일 입출력 예제입니다.</pre>	파일 내용:

③ 하나로 합친 완전한 Python 소스 코드

아래는 “파일로 입력 및 파일로 출력”을 하나로 합친 완전한 Python 소스 코드이다. 이 코드는 파일에 데이터를 쓰고, 저장된 파일을 읽어 출력하는 과정을 포함한다.

주석으로 설명한 소스 코드	출력 결과
<pre> # 파일로 입력 및 파일로 출력 # 파일에 데이터를 쓰고, 저장된 데이터를 읽어 출력하는 코드 # 파일 쓰기 with open("output.txt", "w") as file: file.write("안녕하세요.\n") file.write("Python 파일 입출력 예제입니다.\n") print("데이터가 파일에 저장되었습니다: output.txt") # 파일 읽기 with open("output.txt", "r") as file: content = file.read() print("파일에서 읽은 내용:") print(content) 안녕하세요. Python 파일 입출력 예제입니다. </pre>	output.txt 파일 내용: <pre> 안녕하세요. Python 파일 입출력 예제입니다. </pre>

(1.9.3) $m \times n$ 행렬 [A] 입출력 하기

아래는 "m by n 행렬 [A]의 입출력" 코드를 하나로 통합한 완전한 Python 코드이다. 이 코드는 행렬을 생성하고 파일에 저장한 후, 저장된 파일에서 행렬을 불러와 출력하는 과정을 포함한다. 코드는 간단히 설명하면, 행렬 생성, 파일 저장, 파일 읽기 순으로 이루어져 있다. 행렬 생성은 `np.array()`를 사용하여 NumPy 배열을 3x3 행렬 A로 생성하고 `print()`로 생성된 행렬을 확인한다. 행렬 저장은 `np.savetxt()` 함수를 이용하여 `matrix.txt`에 저장하고, 저장 형식은 `fmt="%d"`로 정수 형식으로 저장하고, `delimiter=","`로 쉼표 데이터로 구분한다. 파일 읽기는 `np.loadtxt()` 함수를 사용하여 `matrix.txt` 파일을 읽어와 NumPy 배열로 복원하고, `dtype=int` 옵션으로 데이터를 정수형으로 변환하였다.

주석으로 설명한 소스 코드	출력 결과
<pre>import numpy as np # m x n 행렬 생성 A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) print("생성된 행렬 A:") print(A) # 행렬을 파일에 저장 np.savetxt("matrix.txt", A, fmt="%d", delimiter=",") print("\n행렬 A가 matrix.txt 파일에 저장.") # 저장된 파일에서 행렬 불러오기 loaded_A = np.loadtxt("matrix.txt", delimiter=",", dtype=int) print("\n파일에서 불러온 행렬 A:") print(loaded_A)</pre>	화면 출력: 생성된 행렬 A: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ 행렬 A가 matrix.txt 파일에 저장. 파일에서 불러온 행렬 A: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ matrix.txt 파일 내용: 1,2,3 4,5,6 7,8,9

1.10 함수(Function)

1.10.1 내장함수와 사용자 정의 함수

내장함수(Intrinsic Functions)는 Python에서 기본적으로 제공하는 함수로, 추가적인 정의 없이 바로 사용할 수 있다. 예를 들어, `len()`, `sum()`, `print()` 등이 있다.

사용자 정의 함수(User-defined Function)는 프로그램의 특정 요구사항을 충족하기 위해 직접 설계한 함수이다. `def` 키워드를 사용하여 정의한다.

1.10.2 파이썬과 NumPy의 내장함수 종류

① Python List 내장함수

- **append**

`append(a)`는 리스트의 마지막에 `a`라는 변수를 추가하는 함수이다.

- **index**

`index(a)`는 리스트에 `a`라는 변수가 있으면 그 값을 `return` 하는 함수이다.

- **insert**

`insert(a,b)`는 리스트의 `a`번째 위치에 `b`를 삽입하는 함수이다.

- **sort**

`sort` 함수는 리스트의 요소를 순서대로 정렬한다.

append, remove, insert 내장 함수 예제

주석으로 설명한 소스 코드	출력 결과
<pre>v=[0, 1, 2, 3] #빈숫자에 리스트를 저장 v.append(111) #v의 리스트에 111을 추가 print(v) #v의 리스트를 출력 v.extend([22, 33, 'aaa']) #새로운 리스트를 추가 print(v) #v의 리스트를 출력 v.remove(22) #v의 리스트에 22를 제거 v.remove(111) #v의 리스트에 111을 제거 v.remove(33) #v의 리스트에 33을 제거 v.remove('aaa')#v의 리스트에 33을 제거 v.remove(1) #v의 리스트에 1을 제거 print(v) #v의 리스트를 출력 v.insert(1,'bbb') #1번에 bbb를 추가 print(v) #v의 리스트를 출력</pre>	<pre>[0, 1, 2, 3, 111] [0, 1, 2, 3, 111, 22, 33, 'aaa'] [0, 2, 3] [0, 'bbb', 2, 3]</pre>

1.10.3 사용자 정의 함수 선언과 특징

Python은 다른 언어(C, C++, Java, Fortran 등)와 마찬가지로 함수를 만들 수 있으며, 이를 통해 특정 작업을 수행하는 코드를 모듈화할 수 있다. 하지만 Python의 함수 선언 방식은 다른 언어들과 몇 가지 차이를 가진다.

1. 함수 선언 방식

- Python에서는 `def` 키워드를 사용하여 함수를 정의한다. `def`는 definition(정의)의 약자로 함수 선언의 시작을 의미한다.
- 함수 이름 뒤에는 괄호가 오며, 괄호 안에는 필요에 따라 인자를 정의할 수 있다. 여러 개의 인자가 있을 경우 쉼표(,)로 구분한다.
- Python은 반환 데이터 유형을 명시적으로 정의하지 않는다. 함수가 값을 반환하려면 `return` 키워드를 사용한다.

2. Python과 다른 언어의 차이점

- Python에서는 Header 파일이 필요하지 않다.
- 변수와 반환값의 데이터 유형을 명시하지 않아도 된다. 이는 C, C++, Java, Fortran 등의 언어와 차이점 중 하나이다.
- Python의 유연한 데이터 관리 방식을 통해 함수 내에서 다양한 데이터 유형을 쉽게 처리할 수 있다.

1.10.4 사용자 정의 함수 예제

1. 함수 예제: 수학적 함수와 도함수

아래는 함수 $f(x)$ 와 그 도함수 $f'(x)$ 를 정의하고 사용하는 Python 코드의 예이다. $f(x)$ 함수는 정의된 수학 함수로, 입력값 x 를 받아 $9 - x(x - 10)$ 의 결과를 반환한다. $fprime(x)$ 도함수는 $f(x)$ 의 도함수로, 입력값 x 에 대해 식 $-2x + 10$ 의 결과를 반환한다. `print()`를 통해 두 함수의 결과를 확인한다.

주석으로 설명한 소스 코드	출력 결과
<pre># f(x) 함수 정의 def f(x): return 9 - x * (x - 10) # f'(x) 도함수 정의 def fprime(x): return -2 * x + 10 # 함수 출력 예제 print("f(3) = ", f(3)) print("f'(3) = ", fprime(3))</pre>	<pre>f(3) = 0 f'(3) = 4</pre>

2. 함수 예제: 팩토리얼 계산

주석으로 설명한 소스 코드	출력 결과
<pre># 사용자 정의 함수: 팩토리얼 계산 def factorial(n): if n == 0 or n == 1: return 1 else: return n * factorial(n - 1) # 사용자 정의 함수 호출 print("5! = ", factorial(5)) print("7! = ", factorial(7))</pre>	<pre>5! = 120 7! = 5040</pre>

3. 함수 예제: 함수 $F(x)$ 를 정의하고 계산

함수 $F(x)$ 를 정의하여 방정식의 값을 계산하여 보자. 주어진 함수 $f(x) = 9 - x(x - 10)$ 에 $x = 101$ 을 대입하여 값을 출력한다. 함수를 정의한 후 변수를 선언하고, 정의한 함수를 호출하여 결과를 출력한다.

주석으로 설명한 소스 코드	출력 결과
<pre>def f(x): return 9 - x * (x - 10) x = 101 print(f(x))</pre>	<pre>-9182</pre>

4. 함수 예제: 사용자 정의 함수와 조건문 결합

`even_or_odd()` 함수는 조건문을 사용하여 입력된 숫자가 짝수인지 홀수인지 판별한다.

주석으로 설명한 소스 코드	출력 결과
<pre># 숫자가 짝수인지 홀수인지 판별하는 함수 def even_or_odd(number): if number % 2 == 0: return f"{number}는 짝수입니다." else: return f"{number}는 홀수입니다." # 함수 호출 print(even_or_odd(10)) print(even_or_odd(7))</pre>	10는 짝수입니다. 7는 홀수입니다.

1.11 NumPy를 이용한 벡터 행렬

1.11.1 내적(Dot Product)과 외적(Cross Product)

1. 벡터의 내적

두 벡터의 대응 요소를 곱하고 그 합을 구하는 연산이다. 내적은 기하학적으로 두 벡터 간의 각도 계산이나 프로젝트에 사용된다. 벡터 간의 유클리드 거리는 두 벡터 사이 벡터의 크기(norm)를 계산한다. n 차원 벡터의 내적 $\mathbf{a} \cdot \mathbf{b}$ 의 공식은 다음과 같다.

$$\mathbf{a} \cdot \mathbf{b} = (a_1b_1) + (a_2b_2) + \cdots + (a_nb_n)$$

$$= \sum_{i=1}^n (a_i b_i)$$

주석으로 설명한 소스 코드	출력 결과
<pre>import numpy as np # 두 벡터 정의 a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) # 내적 계산 dot_product = np.dot(a, b) print("벡터 a와 b의 내적:", dot_product)</pre>	벡터 a와 b의 내적: 32

2. 벡터의 외적

3차원 벡터에서 정의되며, 두 벡터에 수직인 벡터를 반환한다. `np.cross()` 함수는 두 3차원 벡터의 외적을 계산한다. 결과 벡터는 입력 벡터에 수직이다.

주석으로 설명한 소스 코드	출력 결과
<pre># 두 벡터 정의 a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) # 외적 계산 cross_product = np.cross(a, b) print("벡터 a와 b의 외적:", cross_product)</pre>	<pre>벡터 a와 b의 외적: [-3 6 -3]</pre>

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_11_numpy_linear_algebra.py

1.11.2 두 벡터간의 거리 구하기

두 벡터 간의 거리는 일반적으로 유클리드 거리(Euclidean distance)를 사용하여 계산한다. 이는 두 벡터 사이를 벡터로 나타낸 후, 해당 벡터의 크기(norm)를 구하는 방식이다. Python의 NumPy 라이브러리를 사용하면 유클리드 거리를 간단히 계산할 수 있다. n 차원 벡터 $\mathbf{a}, \mathbf{b}$ 의 거리는 다음과 같다.

$$\|\mathbf{a} - \mathbf{b}\| = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_n - b_n)^2}$$

$$= \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

주석으로 설명한 소스 코드	출력 결과
<pre>import numpy as np # 두 벡터 정의 a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) # 유클리드 거리 계산 distance = np.linalg.norm(a - b) print("벡터 a:", a) print("벡터 b:", b) print("두 벡터 간의 거리:", distance)</pre>	<pre>벡터 a: [1 2 3] 벡터 b: [4 5 6] 두 벡터 간의 거리: 5.196152422706632</pre>

1.11.3 두 행렬의 곱 (matrix multiplication)

두 행렬의 곱은 선형대수에서 중요한 연산으로, NumPy에서는 @ 연산자 또는 np.dot()을 사용하여 계산할 수 있다. @ 연산자는 Python 3.5 이상에서 도입된 행렬 곱(matrix multiplication) 연산자이며 2차원 배열(행렬)에 대해 적용된다. np.dot()은 NumPy에서 제공하는 함수로, 다목적 내적 연산을 수행한다. 즉, 벡터와 행렬 모두 지원하며, 다차원 배열의 내적 연산도 수행할 수 있다.

연산자 / 함수	기능	적용 범위	예시
@	행렬 곱 전용	2차원 배열(행렬)	$C = A @ B$
np.dot()	다목적 내적 연산	1차원(벡터), 2차원(행렬), 다차원 배열	$C = \text{np.dot}(A, B)$

행렬 연산은 반복 계산의 대표적인 예이다. Python에서는 NumPy 라이브러리를 활용하여 효율적으로 행렬과 관련된 반복 계산을 수행할 수 있다.

NumPy를 이용한 행렬 곱

일반적인 $N \times N$ 행렬 A, B 에 대해 행렬의 곱을 행렬 C 라 하면,

$$C_{ij} = \sum_{k=1}^N A_{ik} B_{kj} \quad (i, j = 1, \dots, N) \quad (1.11.3-1)$$

과 같이 나타낼 수 있다. 두 행렬 A 와 B 의 행렬 곱을 계산할 때, NumPy는 벡터 연산을 지원하므로 반복문 없이 간단히 행렬 곱을 수행할 수 있다. @ 연산자나 np.dot() 함수는 동일한 결과를 반환한다.

** 연산자로 행렬 곱 (AB)

주석으로 설명한 소스 코드	출력 결과
<pre># numpy 라이브러리 импорт import numpy as np # 행렬 선언 A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]]) # 행렬 곱 수행 (np.matmul 사용) C = np.matmul(A, B) print("C =", C) # 행렬을 NumPy의 matrix 객체로 변환 A1 = np.asmatrix(A) print("A1 =", A1) B1 = np.asmatrix(B) print("B1 =", B1) # matrix 객체에서 '@' 연산자로 행렬 곱 수행 C1 = A1 * B1 print("C1 =", C1)</pre>	<pre>C = [[19 22] [43 50]] A1 = [[1 2] [3 4]] B1 = [[5 6] [7 8]] C1 = [[19 22] [43 50]]</pre>

np.dot(A,B) 또는 '@' 연산자로 행렬 곱 (AB)

주석으로 설명한 소스 코드	출력 결과
<pre>import numpy as np # 행렬 선언 A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]]) # 행렬 곱 계산 C = A @ B # 또는 np.dot(A, B) print("행렬 곱 결과:\n", C)</pre>	<pre>행렬 곱 결과: [[19 22] [43 50]]</pre>

numpy 를 사용하지 않은 행렬 곱 (AB)

주석으로 설명한 소스 코드	출력 결과
<pre> # 2차원 배열을 생성하는 함수 정의 def A_0(rows, cols): A = [] # 빈 리스트 초기화 while len(A) < rows: # 행 개 수 가 rows 보 다 작으면 반복 A.append([]) # 새로운 행 추가 # 열 개수가 cols보다 작으면 반복 while len(A[-1]) < cols: A[-1].append(0.0) # 행에 0.0 추가 return A # 생성된 2차원 배열 반환 # 두 행렬의 곱을 계산하는 함수 정의 def AmultiB(A, B): rowsA = len(A) # A 행렬의 행 개수 colsA = len(A[0]) # A 행렬의 열 개수 rowsB = len(B) # B 행렬의 행 개수 colsB = len(B[0]) # B 행렬의 열 개수 # 행렬 곱이 가능한지 확인 (A의 열 개수 == B 의 행 개수) if colsA != rowsB: # 조건이 맞지 않으면 예외 발생 raise ArithmeticError('colsA != rowsB') # 결과 행렬의 크기는 rowsA x colsB AmultiB = A_0(rowsA, colsB) # 행렬 곱 계산 for i in range(rowsA): # A의 행 반복 for j in range(colsB): # B의 열 반복 sum = 0 # 곱셈과 덧셈의 결과 저 장 변수 # A의 열과 B의 행 반복 for ii in range(colsA): sum = sum + A[i][ii] * B[ii][j] # 결과 행렬의 (i, j) 위치에 저장 AmultiB[i][j] = sum return AmultiB # 결과 행렬 반환 # 행렬 선언 A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]]) # A와 B의 곱 계산 C = AmultiB(A, B) print('C =', C) </pre>	<pre> C = [[19, 22], [43, 50]] </pre>

1.12 객체지향 프로그래밍(OOP)

1.12.1 클래스 개요

OOP는 객체 지향 프로그래밍으로 프로그램을 구성하는 데이터와 기능을 논리적으로 묶어 하나의 객체(Object)로 표현한다. 객체는 데이터와 기능을 모두 포함하는데, 데이터는 속성이라 하며, 기능은 메서드라고 한다. 객체는 클래스를 템플릿으로 생성되며, 같은 클래스에서 생성된 객체들은 공통된 속성과 기능을 가진다. 파이썬은 객체 지향 프로그래밍을 지원하는 언어로, 모든 것이 객체이다. 따라서 파이썬에서는 클래스를 정의하여 객체를 생성하고, 객체의 속성과 메서드를 사용할 수 있다.

객체 지향 프로그래밍은 다음과 같은 특징을 가진다.

① 캡슐화(Encapsulation):

객체의 속성과 메서드를 하나로 묶어 정보 은닉을 구현한다. 객체 외부에서는 객체 내부의 데이터에 직접적으로 접근할 수 없으며, 메서드를 통해 간접적으로 접근하게 된다.

② 상속(Inheritance):

부모 클래스의 특징을 자식 클래스가 물려받는 것으로, 코드의 재사용성을 높인다.

③ 다형성(Polymorphism):

같은 이름의 메서드를 사용하여 객체마다 다른 동작을 할 수 있도록 한다.

1.12.2 클래스 선언

파이썬에서 클래스는 `class` 키워드를 사용하여 선언한다. 클래스는 속성과 메서드를 포함하며, 객체지향 프로그래밍의 핵심 요소로 데이터와 동작을 하나의 단위로 묶어 관리할 수 있다.

1.12.3 속성(변수)

① 인스턴스 변수

인스턴스 변수는 각 인스턴스마다 독립된 변수이다. 예를 들어, 클래스는 여러 개의 인스턴스를 생성할 수 있으며, 각각의 인스턴스 변수는 서로 다른 값으로 설정된다. 인스턴스마다 각기 다른 값을 저장할 수 있다.

② 클래스 변수

클래스 변수는 클래스의 모든 인스턴스 사이에 공유되는 값을 가진 변수이다. 클래스 변수는 인스턴스를 생성하지 않고도 접근하거나 참조할 수 있다.

1.12.4 클래스 메서드(함수)

클래스 메서드는 클래스 자체를 대상으로 동작한다. 클래스 레벨에서 데이터를 다루는 메서드이다. 일반 메서드가 특정 객체를 대상으로 동작하는 것과 달리, 클래스 메서드는 클래스 자체를 첫 번째 인자로 받아 클래스의 속성과 동작을 조작하거나 액세스할 수 있다.

파이썬에서는 `@classmethod` 데코레이터를 사용하여 정의한다. 첫 번째 매개변수는 `cls`를 사용한다. `cls`는 클래스 자체를 참조하며 이를 통해 클래스 속성에 접근하거나 새로운 객체를 생성할 수 있다.

클래스 메서드는 객체와 클래스 레벨의 상호작용을 효율적으로 관리하기 위한 강력한 도구이다.

① 클래스 메서드의 주요 특징

- **클래스 레벨에서 동작:** 클래스 변수를 수정하거나 클래스 자체의 동작을 정의하는데 사용한다.
- **클래스 이름으로 직접 호출 가능:** 인스턴스를 생성하지 않고도 클래스 이름을 통해 호출할 수 있다.
- **데이터 관리 및 팩토리 메서드 역할:** 클래스와 관련된 데이터를 관리하거나 객체를 생성하는 팩토리(factory) 메서드로 활용된다.

② 예제 코드

Class methods operate at the class level:

```
class Example:
    class_variable = 0

    def __init__(self, value):
        self.instance_variable = value

    @classmethod
    def increment_class_variable(cls, amount):
        cls.class_variable += amount
        print(f"Class variable updated to {cls.class_variable}")

    @classmethod
    def create_instance(cls, value):
        return cls(value)

# Usage
Example.increment_class_variable(10)
# Output: Class variable updated to 10

instance = Example.create_instance(20)
print(instance.instance_variable) # Output: 20
```

1.12.5 객체지향 프로그래밍(OOP) 예제 1

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_12_oop.py

```
# Define class
class Animal:
    def __init__(self, name): # Constructor
        self.name = name

    def speak(self):
        print(f"{self.name} is making a sound.")

# Inheritance
class Dog(Animal):
    def speak(self): # Method overriding (polymorphism)
```

```

        print(f"{self.name} says Woof!")

# Create object and use
dog = Dog("Buddy")
dog.speak()
# Output: Buddy says Woof!

```

1.12.6 객체지향 프로그래밍(OOP) 예제 2

이 코드는 객체지향 프로그래밍(OOP)을 활용하여 함수와 미분을 관리하고 시각화하는 예제이다. 이 코드는 객체지향 프로그래밍(OOP)을 사용하여 함수의 기본을 정의하고 가시화하는 프로그램이다. Function 클래스는 함수와 그 미분을 저장하며, 주어진 입력 값에 함수 값을 계산하는 `evaluate` 메서드와 미분 값을 계산하는 `derivative` 메서드를 포함한다. `visualize` 메서드는 지정된 범위에서 함수와 미분의 그래프를 그리는 기능을 제공한다. 이 코드에서는 함수 $f(x) = x^2$ 과 미분 $f'(x) = 2x$ 를 정의하고, 이들 함수를 객체로 생성하여 시각화한다. 그래프는 x 의 범위를 -10에서 10까지로 설정하며, 파란색 선으로 함수 $f(x) = x^2$, 주황색 점선으로 미분 $f'(x) = 2x$ 를 나타낸다. 결과적으로 함수와 미분의 관계를 시각적으로 확인할 수 있는 도구를 제공한다.

실습예제 https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_12_06_oop_ex2.py

1.13 matplotlib를 이용한 그래프 시각화

Overview

이 절에서는 파이썬의 데이터 시각화 라이브러리인 Matplotlib를 사용하여 데이터를 그래프로 표현하는 방법을 설명한다. Matplotlib는 선 그래프, 막대 그래프, 히스토그램, 산점도 등 다양한 그래프 유형을 지원하며, 데이터를 시각적으로 분석하는 데 유용하다. 그래프의 기본 구성 요소인 제목, 축 레이블, 범례 등을 설정하여 그래프를 사용자 정의할 수 있다. 또한, 플롯 스타일을 조정하거나, 서브플롯을 생성해 여러 데이터를 동시에 시각화하는 방법도 다룬다. 이 절에서는 Matplotlib의 기본 사용법부터 고급 그래프 작성 방법까지 실습과 함께 다루며, 데이터를 효과적으로 표현하고 분석할 수 있는 방법을 학습한다.

Supported Graph Types:

- Line graphs
- Bar charts
- Histograms
- Scatter plots
- 3D surface plots

Key Components:

- Title, axis labels, legends
- Plot styles and colors
- Subplots for multiple visualizations

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_13_matplotlib.py

1.13.1 그래프 그리기 위한 주요 내장함수

Matplotlib에서 자주 사용하는 주요 함수들은 다음과 같다:

- `plot()`: 선 그래프를 그리는 함수.
- `scatter()`: 산점도를 그리는 함수.
- `bar()`: 막대 그래프를 그리는 함수.
- `hist()`: 히스토그램을 그리는 함수.
- `imshow()`: 행렬 데이터를 시각화하는 함수.
- `show()`: 그래프를 화면에 출력.

1.13.2 2차원 그래프

2D Line Graph Example

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
x = np.linspace(-10, 10, 500)
y = x**2

# Create graph
plt.figure(figsize=(10, 6))
plt.plot(x, y, label='f(x) = x^2', color='blue', linewidth=2)

# Add annotations
plt.title('Line Graph of f(x) = x^2', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('f(x)', fontsize=12)
plt.axhline(0, color='black', linestyle='-', linewidth=0.8)
plt.axvline(0, color='black', linestyle='-', linewidth=0.8)
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(fontsize=12)

plt.show()
```

1.13.3 2차원 산점도

이 코드는 x, y 의 관계를 시각화한 2차원 산점도를 그래프로 생성한다. x 는 0에서 10 사이의 100개의 난수이며, $y = 2x + noise$ 로 계산된 값으로 각 점을 파란색으로 표시하고, 제목, 축 레이블, 격자선, 범례를 추가하여 데이터 분포를 직관적으로 표현한다.

2D Scatter Plot

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
np.random.seed(42)
x = np.random.rand(100) * 10
y = 2 * x + np.random.normal(0, 3, 100)

# Create scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='blue', alpha=0.7, label='Data points')

# Add annotations
plt.title('2D Scatter Plot', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(fontsize=12)

plt.show()
```

1.13.4 3차원 그래프

이 코드는

$$z = \sin\left(\sqrt{x^2 + y^2}\right)$$

을 3D 표면 그래프로 시각화한다.

x 와 y 는 -5에서 5까지의 값을 균등 분할하여 생성하고, `np.meshgrid`를 사용해 격자(grid) 형태의 좌표를 만든다. z 값은 함수 $\sin(\sqrt{x^2 + y^2})$ 를 통해 계산된다.

3D 그래프는 `plot_surface`로 생성하며, 색상은 `viridis` 컬러맵을 적용한다. 제목과 축 레이블을 추가하여 그래프의 가독성을 높인다.

3D Surface Plot

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import matplotlib.pyplot as plt

# Prepare data
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

# Create 3D graph
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, Y, Z, cmap="viridis")

ax.set_title("3D surface")
```

```
ax.set_xlabel("X axis")
ax.set_ylabel("Y axis")
ax.set_zlabel("Z axis")

plt.show()
```

1.14 연습문제